Optimizing Predictive Maintenance For Industrial IoT Devices To Reduce Operational Costs and Downtime

IITM BS Degree Program,

Indian Institute of Technology, Madras, Chennai,

Team: IndustryGeeks

Course: Industry 4.0 | IIT Madras BS

Leader:

Diya Nathwani - 21f1003794@ds.study.iitm.ac.in

Members:

Diya Nathwani - 21f1003794@ds.study.iitm.ac.in
Sarthak Singh Gaur - 21f3001936@ds.study.iitm.ac.in
Tasneem Shahnawaz - 22f2001653@ds.study.iitm.ac.in
Pawan Solanki - 22f2000033@ds.study.iitm.ac.in
Bhabatosh Acharya - 22f2000802@ds.study.iitm.ac.in

1. PROBLEM IDENTIFICATION & DEFINITION

1.1 Background

Industrial IoT systems generate continuous sensor data from distributed devices. Reactive maintenance leads to unexpected failures and high downtime costs. This project develops a predictive maintenance framework to forecast failures and enable proactive interventions.

1.2 Problem Statement

The company operates 1,169 IoT-enabled devices with historical logs of 124,494 device-time records and a binary failure indicator, but only 106 records correspond to failures (~0.085%), creating an extreme class-imbalance problem. The core problem is to predict impending device failures early enough to act, while controlling both missed failures (false negatives) and unnecessary maintenance actions (false positives).

1.3 Objective

The objective is to build a predictive maintenance framework that estimates failure probability at each device-timestamp and supports proactive scheduling of maintenance. The solution explicitly incorporates business costs to minimize total operational cost by balancing expensive missed failures against cheaper, but numerous, unnecessary maintenance actions.

1.4 Decision Variables

Key decision variables are:

(i) predicted failure probability per device-time instance,
(ii) the decision threshold above which a device is classified as “maintenance required,” and
(iii) the resulting maintenance action (trigger preventive intervention vs. continue normal operation).

2. CONSTRAINTS

2.1 Data Constraints

Failures are extremely rare (106 failures in 124,494 records, ~0.085%), making naive accuracy-based models misleading. Failures occur across a subset of devices (~106 devices experienced failures), while most devices do not fail during the observation period., so most devices never fail in the observation window, limiting per-device failure history and emphasizing the need for temporal sensor patterns rather than device ID.

2.2 Operational Constraints

Missing a true failure can cause unplanned downtime and high economic loss, so false negatives carry a large penalty in the cost function. However, excessive false positives would lead to many unnecessary inspections and maintenance operations, which is also financially undesirable, so the model must operate at an economically optimal trade-off instead of maximizing raw accuracy.

3. RELEVANT DATA

3.1 Dataset Overview

The dataset consists of 124,494 rows and 12 original columns: date, device ID, a binary failure flag, and nine numeric sensor metrics (metric1-metric9). There are 1,169 unique devices and 106 recorded failure events spread sporadically over a 10-month period (January-November 2015).

3.2 Input Features

Input features include the nine sensor metrics capturing device operating conditions, calendar variables (day, month, weekday) derived from the timestamp, and lag-based features that encode recent sensor history. For each device, lag1 and lag3 values were generated for all nine metrics to capture pre-failure degradation patterns.

3.3 Target Variable

The target variable is a binary failure indicator, where 1 denotes a device failure at that timestamp and 0 denotes normal

operation. Given the extremely low prevalence of failures, this is a rare-event classification task that requires cost and imbalance-aware modeling.

3.4 Data Challenges

Extreme class imbalance means a trivial “always healthy” model appears highly accurate but is useless for maintenance decisions. Failures are isolated at the device level and many metrics stay near zero for long periods, with strong deviations only near failures, which reinforces the need for temporal and multivariate feature engineering instead of simple static thresholds.

4. APPROACH & METHODOLOGY

- **Step 1 - Data Preprocessing**
  Dates were converted from string to datetime, and calendar components (day, month, weekday) were extracted for analysis. The dataset was sorted by device and date to preserve temporal order, and non-numeric identifiers (date, device) were removed from the modeling matrix while kept for interpretation, resulting in a modeling dataset of 120,989 records.
- **Step 2 - Feature Engineering**
  For each device, lag1 and lag3 features were created for metric1-metric9 using grouped shifts, and rows with missing lags were dropped. Analysis showed observable differences in pre-failure behavior, for example metric2_lag1 averaged about 3,203 before failures versus roughly 146 in normal records, and metric4_lag1 about 47 versus 1.6, indicating potential early warning patterns 1-3 time steps before failure.
- **Step 3 - Model Development**
  The processed data was split into training and test sets using an 80-20 stratified split to preserve the rare failure proportion. Two models were trained: Logistic Regression and Random Forest, both configured with class_weight="balanced" to partially counter the severe class imbalance.
- **Step 4 - Model Evaluation**

To reflect real economics, a custom cost function was defined:

Total Cost = (False Negatives × 1,000 units) + (False Positives × 10 units)

Predicted probabilities were swept over 100 thresholds to find the minimum-cost operating point.

At the optimal threshold of approximately 0.616 for Logistic Regression, the test-set confusion profile is 451 false positives and 9 false negatives, yielding a total cost of 13,510 units; the untuned baseline cost was about 41,720 units, so modeled operational cost reduced by roughly 68%. Random Forest produced a slightly higher minimum cost with lower recall, making Logistic Regression the preferred model.

Cost-Optimized Model Comparison

Model	Threshold	Recall (Failure)	False Positives	Total Cost (units)
Logistic Regression	0.616	0.57	451	13,510
Random Forest	0.020	0.38	96	13,960

- **Step 5 - Deployment Concept**
  In deployment, the selected model would periodically score each device and compute a failure probability in near real time. Devices whose predicted risk exceeds the optimized threshold (around 0.616) would be flagged for inspection or scheduled maintenance, integrating with a Computerized Maintenance Management System (CMMS) to prioritize work orders by predicted risk and cost impact.

5. EXPECTED OUTCOMES

The cost-sensitive Logistic Regression model with threshold optimization is selected as the final model, achieving strong performance at an operating threshold of about 0.616. Under the defined cost function (1,000 units per false negative, 10 per false positive), the optimized configuration yields a total cost of 13,510 units versus approximately 41,720 units before threshold tuning, corresponding to about a 68% reduction in modeled operational cost. Random Forest delivers slightly higher cost (~13,960 units) and lower recall, so Logistic Regression offers the better cost-recall trade-off.